

CS-112

Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

COURSE INSTRUCTOR : MS. HABIBA ARSHAD

File Handling in Java

File Handling

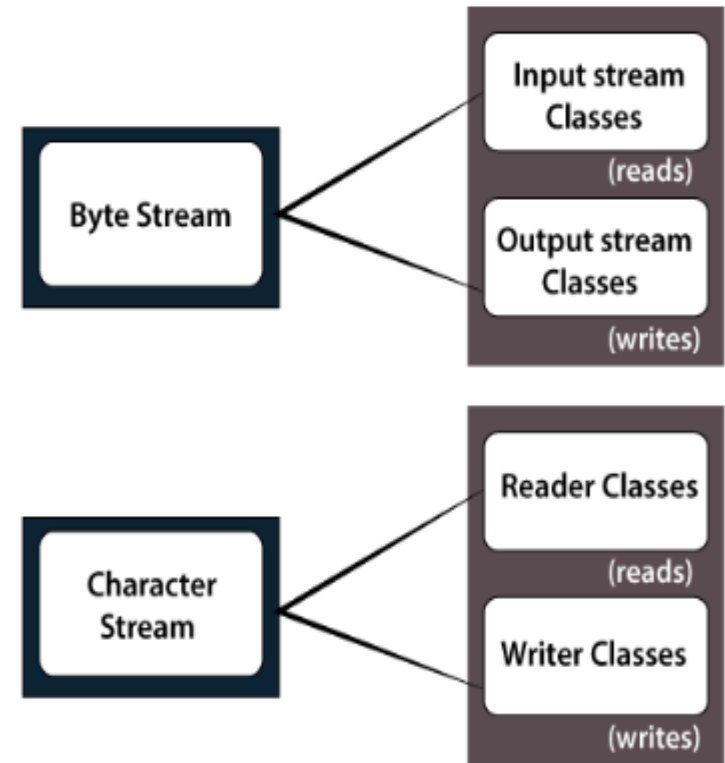
1. File handling in Java is defined as reading and writing data to a file.
2. The particular file class import from the package called **java.io** allows us to handle and work with different formats of files.
3. Thus, if we want to use a file class, we need to create an object of that particular class and should specify the filename or directory name.

File Operations in Java

1. In Java, a **File** is an abstract data type.
2. A named location used to store related information is known as a **File**.
3. There are several **File Operations** like **creating a new File**, **getting information about File**, **writing into a File**, **reading from a File** and **deleting a File**.
4. Before understanding the File operations, it is required that we should have knowledge of **Stream** and **File methods**.

Stream

1. A series/sequence of data is referred to as a **stream**.
2. On the basis of java.io package, **Stream** is classified into two types, i.e., **Byte Stream** and **Character Stream**.



Brief classification of I/O streams

Byte Stream

1. **Byte Stream** is mainly involved with byte data.
2. A file handling process with a byte stream is a process in which an input is provided and executed with the byte data.
3. Data read in byte stream is 8 bytes at a time

Character Stream

1. **Character Stream** is mainly involved with character data.
2. A file handling process with a character stream is a process in which an input is provided and executed with the character data.
3. Data read in character stream is character by character

Java.io package

- Classes related to input and output are present in the Java™ language package `java.io` .
- Java technology uses "streams" as a general mechanism of handling data.
- Input streams act as a source of data.
- Output streams act as a destination of data.

S.No.	Method	Return Type	Description
1.	canRead()	Boolean	The canRead() method is used to check whether we can read the data of the file or not.
2.	createNewFile()	Boolean	The createNewFile() method is used to create a new empty file.
3.	canWrite()	Boolean	The canWrite() method is used to check whether we can write the data into the file or not.
4.	exists()	Boolean	The exists() method is used to check whether the specified file is present or not.
5.	delete()	Boolean	The delete() method is used to delete a file.
6.	getName()	String	The getName() method is used to find the file name.
7.	getAbsolutePath()	String	The getAbsolutePath() method is used to get the absolute pathname of the file.
8.	length()	Long	The length() method is used to get the size of the file in bytes.
9.	list()	String[]	The list() method is used to get an array of the files available in the directory.
10.	mkdir()	Boolean	The mkdir() method is used for creating a new directory.

Java File Class Methods

File Handling Operations

Some of the common file handling operations are;



Create File

1. **Create a File** operation is performed to create a new file.
2. We use the **createNewFile()** method of file.
3. The **createNewFile()** method returns true when it successfully creates a new file and returns false when the file already exists.
4. Let's take an example of creating a file to understand how we can use the **createNewFile()** method to perform this operation.

```
import java.io.File; // Import the File class

File myObj = new File("filename.txt"); // Specify the filename
```

```
// Importing File class
import java.io.File;

// Importing the IOException class for handling errors
import java.io.IOException;

class CreateFile {
    public static void main(String args[]) {
        try {
            // Creating an object of a file
            File f0 = new File("D:FileOperationExample.txt");
            if (f0.createNewFile()) {
                System.out.println("File " + f0.getName() + " is created successfully.");
            } else {
                System.out.println("File is already exist in the directory.");
            }
        } catch (IOException exception) {
            System.out.println("An unexpected error is occurred.");
            exception.printStackTrace();
        }
    }
}
```

Example: CreateFile.java

In the given code, we import the File and IOException class for performing file operation and handling errors, respectively.

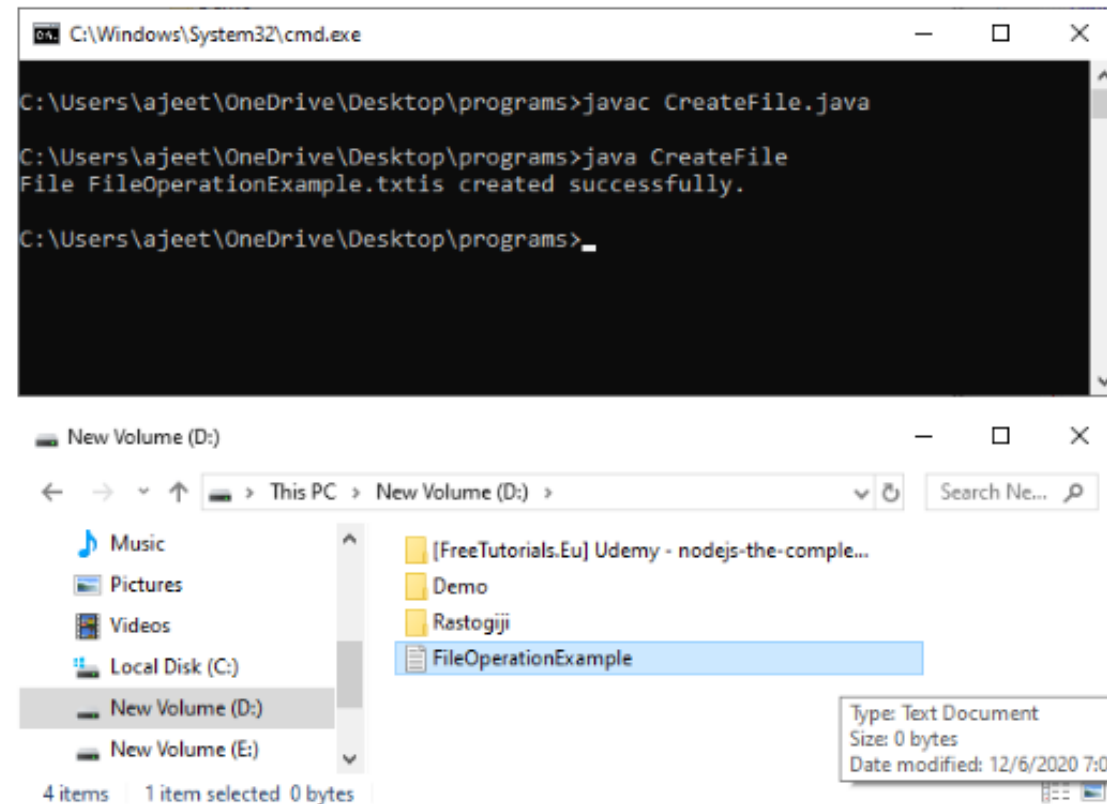
We create the **f0** object of the File class and specify the location of the directory where we want to create a file.

In the try block, we call the **createNewFile()** method through the **f0** object to create a new file in the specified location.

If the method returns false, it will jump to the else section. If there is any error, it gets handled in the catch block.

Output

Output:



Get File Information

The operation is performed to get the file information.

We use several methods to get the information about the file like name, absolute path, is readable, is writable and length.

Let's take an example to understand how to use file methods to get the information of the file.

Example: FileInfo.java

```
// Import the File class
import java.io.File;

class FileInfo {
    public static void main(String[] args) {
        // Creating file object
        File f0 = new File("D:FileOperationExample.txt");
        if (f0.exists()) {
            // Getting file name
            System.out.println("The name of the file is: " + f0.getName());

            // Getting path of the file
            System.out.println("The absolute path of the file is: " + f0.getAbsolutePath());

            // Checking whether the file is writable or not
            System.out.println("Is file writeable?: " + f0.canWrite());

            // Checking whether the file is readable or not
            System.out.println("Is file readable " + f0.canRead());

            // Getting the length of the file in bytes
            System.out.println("The size of the file in bytes is: " + f0.length());
        } else {
            System.out.println("The file does not exist.");
        }
    }
}
```

Output

```
The name of the file is: FileOperationExample.txt  
The absolute path of the file is: D:\\FileOperationExample.txt  
Is file writeable?: true  
Is file readable true  
The size of the file in bytes is: 0
```


Write to a File

The next operation which we can perform on a file is "**writing into a file**".

In order to write data into a file, we will use the **FileWriter class** and its **write()** method together.

We need to close the stream using the **close()** method to retrieve the allocated resources.

Let's take an example to understand how we can write data into a file.

```
// Importing the FileWriter class
import java.io.FileWriter;

// Importing the IOException class for handling errors
import java.io.IOException;

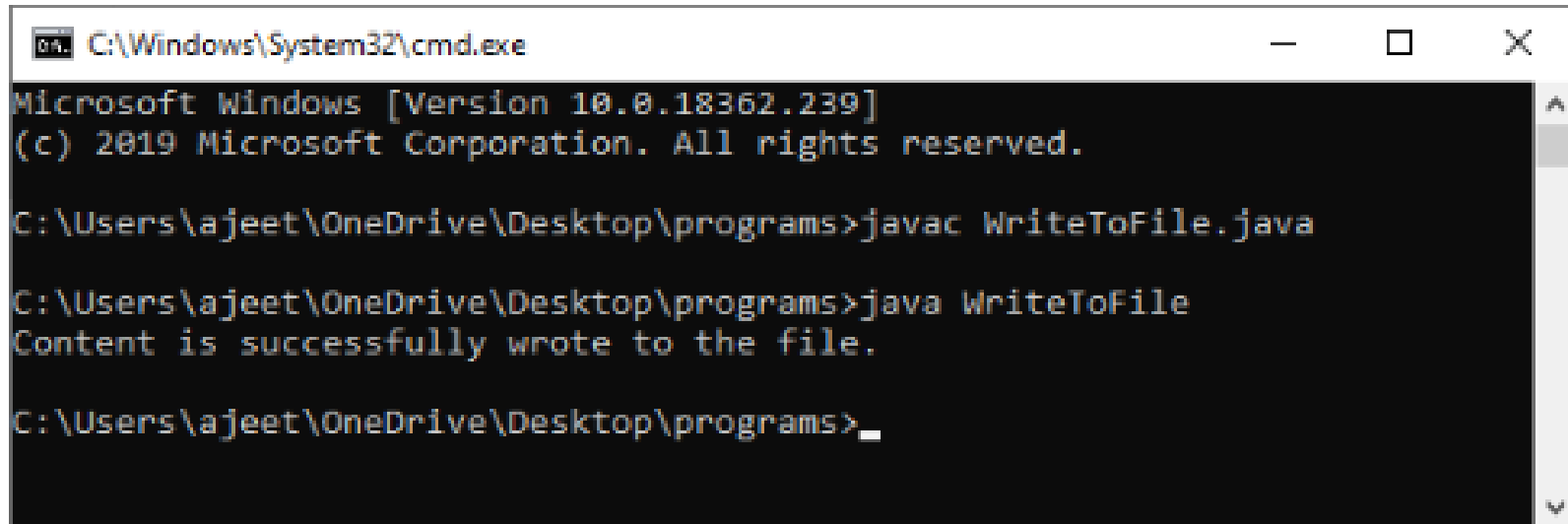
class WriteToFile {
    public static void main(String[] args) {

        try {
            FileWriter fwrite = new FileWriter("D:FileOperationExample.txt");
            // writing the content into the FileOperationExample.txt file
            fwrite.write("A named location used to store related information is referred to as a File.");

            // Closing the stream
            fwrite.close();
            System.out.println("Content is successfully wrote to the file.");
        } catch (IOException e) {
            System.out.println("Unexpected error occurred");
            e.printStackTrace();
        }
    }
}
```

Example: WriteToFile.java

Output:

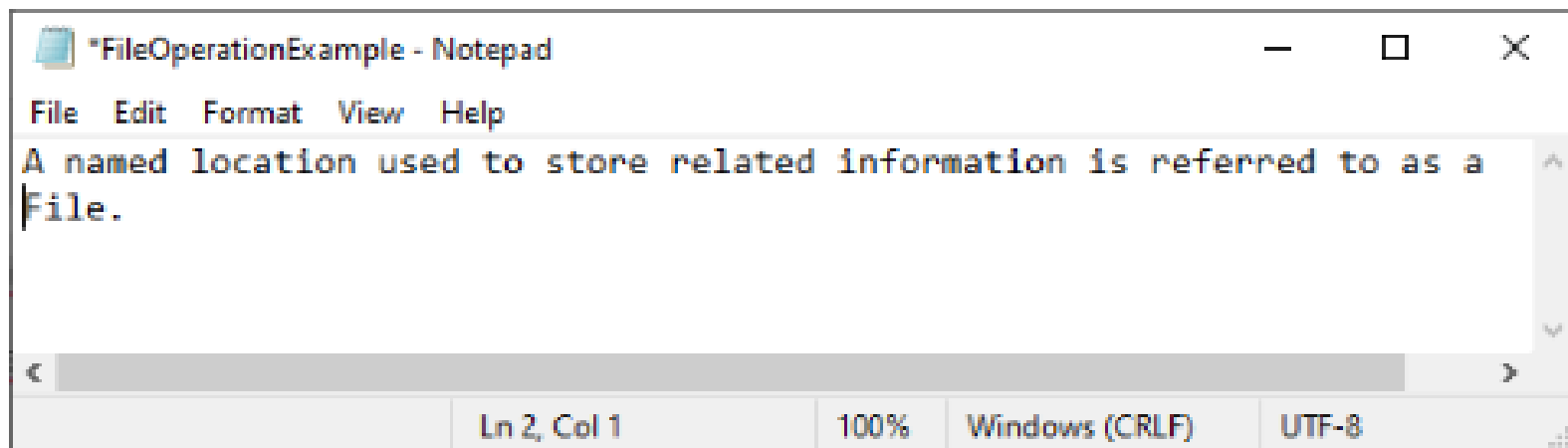


```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.18362.239]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\ajeet\OneDrive\Desktop\programs>javac WriteToFile.java

C:\Users\ajeet\OneDrive\Desktop\programs>java WriteToFile
Content is successfully wrote to the file.

C:\Users\ajeet\OneDrive\Desktop\programs>_
```



```
*FileOperationExample - Notepad
File Edit Format View Help
A named location used to store related information is referred to as a
File.
```

Ln 2, Col 1 100% Windows (CRLF) UTF-8

Read from a File

The next operation which we can perform on a file is "read from a file".

In order to write data, we will use the **Scanner** class.

Here, we need to close the stream using the **close()** method.

We will create an instance of the Scanner class and use the **hasNextLine()** method **nextLine()** method to get data from the file.

Let's take an example to understand how we can read data from a file.

Example ReadFromFile.java

```
// Importing the File class
import java.io.File;

// Importing FileNotFoundException class for handling errors
import java.io.FileNotFoundException;

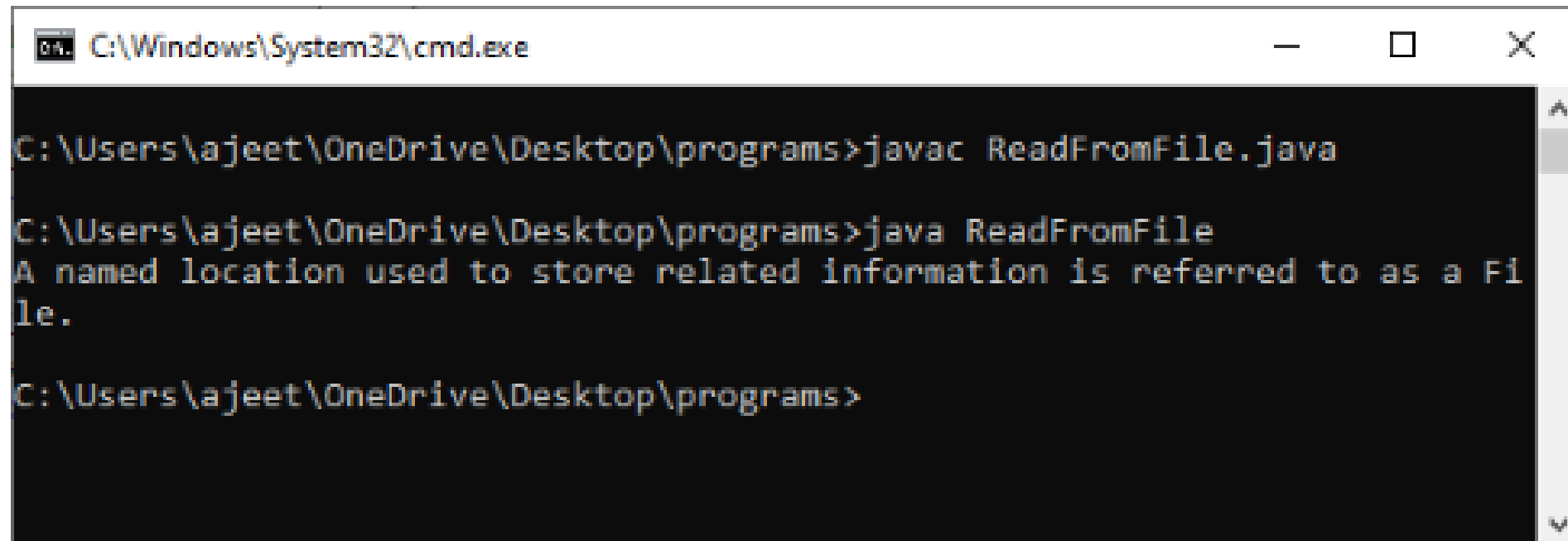
// Importing the Scanner class for reading text files
import java.util.Scanner;

class ReadFromFile {
    public static void main(String[] args) {
        try {
            // Create f1 object of the file to read data
            File f1 = new File("D:FileOperationExample.txt");
            Scanner dataReader = new Scanner(f1);
```

```
            while (dataReader.hasNextLine()) {
                String fileData = dataReader.nextLine();
                System.out.println(fileData);
            }
            dataReader.close();
        } catch (FileNotFoundException exception) {
            System.out.println("Unexpected error occurred!");
            exception.printStackTrace();
        }
    }
}
```

Output

Output:



```
C:\Windows\System32\cmd.exe

C:\Users\ajeet\OneDrive\Desktop\programs>javac ReadFromFile.java

C:\Users\ajeet\OneDrive\Desktop\programs>java ReadFromFile
A named location used to store related information is referred to as a File.

C:\Users\ajeet\OneDrive\Desktop\programs>
```

Delete a File

The next operation which we can perform on a file is "**deleting a file**".

In order to delete a file, we will use the **delete()** method of the file.

We don't need to close the stream using the **close()** method because for deleting a file, we neither use the FileWriter class nor the Scanner class.

Let's take an example to understand how we can write data into a file.

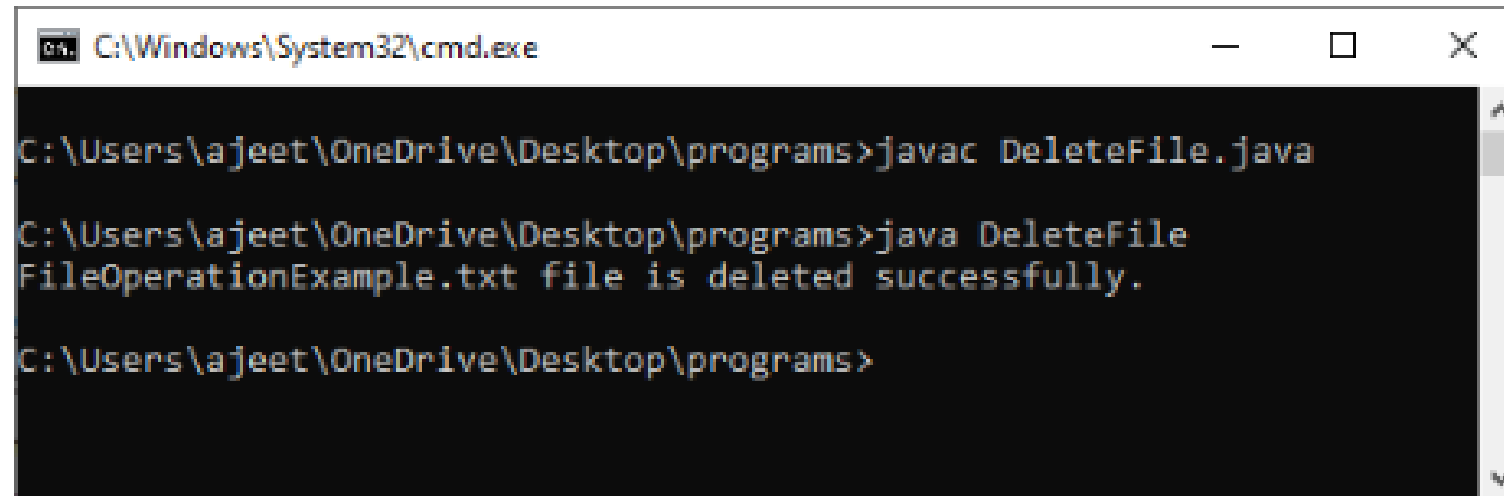
```
// Importing the File class
import java.io.File;

class DeleteFile {

    public static void main(String[] args) {
        File f0 = new File("D:FileOperationExample.txt");
        if (f0.delete()) {
            System.out.println(f0.getName() + " file is deleted successfully.");
        } else {
            System.out.println("Unexpected error found in deletion of the file.");
        }
    }
}
```

Example:DeleteFile.java

Output



```
C:\Windows\System32\cmd.exe

C:\Users\ajeet\OneDrive\Desktop\programs>javac DeleteFile.java

C:\Users\ajeet\OneDrive\Desktop\programs>java DeleteFile
FileOperationExample.txt file is deleted successfully.

C:\Users\ajeet\OneDrive\Desktop\programs>
```

This program depicts how to create a text file using FileWriter,

Java



```
// Creating a text File using FileWriter
import java.io.FileWriter;
import java.io.IOException;
class CreateFile
{
    public static void main(String[] args) throws IOException
    {
        // Accept a string
        String str = "File Handling in Java using "+
            " FileWriter and FileReader";

        // attach a file to FileWriter
        FileWriter fw=new FileWriter("output.txt");

        // read character wise from string and write
        // into FileWriter
        for (int i = 0; i < str.length(); i++)
            fw.write(str.charAt(i));

        System.out.println("Writing successful");
        //close the file
        fw.close();
    }
}
```

This program depicts how to read from the 'text' file using FileReader

Java

```
// Reading data from a file using FileReader
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
class ReadFile
{
    public static void main(String[] args) throws IOException
    {
        // variable declaration
        int ch;

        // check if File exists or not
        FileReader fr=null;
        try
        {
            fr = new FileReader("text");
        }
        catch (FileNotFoundException fe)
        {
            System.out.println("File not found");
        }

        // read from FileReader till the end of file
        while ((ch=fr.read())!=-1)
            System.out.print((char)ch);

        // close the file
        fr.close();
    }
}
```

Tasks: (Read, discuss in-groups and understand about them. (as part of course))

1. Java program to read content from one file and write it into another file.
2. Java program to get file creation, last access and last modification time
3. Java program to get the size of give file in bytes, kilobytes and megabytes
4. Java program to **copy files**.
5. How can you read files using tokenizers in java.